Eclipse Mosquitto

Technical Documentation Guide

Prepared from the official documentation index and linked Mosquitto manuals

Official documentation: https://mosquitto.org/documentation/

Purpose

A practical, structured guide to Mosquitto and MQTT: broker concepts, topic naming, publishing, subscribing, configuration, retained messages, QoS, security, troubleshooting, and integrating an MQTT subscriber with a backend REST API.

Scope note: This PDF is an original explanatory guide and reference map based on the official Mosquitto documentation. It is not a verbatim copy of every official manual page or a replacement for version-specific options in the live documentation.

1. Official documentation index

The official index links to broker and configuration manuals, command-line tools, MQTT behavior, listener guidance, persistence, plugins, authentication, migration notes, and the libmosquitto API.

Document	Purpose
Main documentation index	Index of official manuals and guides
mosquitto — broker	Running the broker
mosquitto.conf	Configuration reference
mosquitto_ctrl	Broker management utility; check the index for current links
mosquitto_passwd	Password-file utility
mosquitto_pub	Publish messages
mosquitto_rr	Simple request/response utility
mosquitto_sub	Subscribe to topics
MQTT manual	MQTT concepts and behavior
Authentication methods	Authentication options
Dynamic Security	Dynamic authentication and access control
libmosquitto API	Client library API reference; use official index if the URL changes

Because documentation URLs and available options can change by Mosquitto version, use the official index as the authoritative starting point.

2. What Mosquitto and MQTT are

Mosquitto is an MQTT broker. **MQTT** is a lightweight publish/subscribe messaging protocol. Clients connect to a broker; publishers send messages to topics; subscribers receive messages matching their subscriptions. Publisher and subscriber do not need a direct connection to each other.

```
Publisher (meter/device/backend)
          |
          | publish(topic, payload)
          v
     Mosquitto broker
          |
          | routes matching messages
          v
Subscriber (backend/dashboard/another device)
```

Common terms

Term	Meaning
Broker	Server that accepts MQTT connections and routes messages.
Client	Any device or application connected to the broker.
Publisher	Client that sends a message to a topic.
Subscriber	Client that registers interest in one or more topic filters.
Topic	Hierarchical routing name, separated by forward slashes.
Payload	Message content, such as JSON, text, or binary data.
QoS	Quality of Service delivery level.

Term	Meaning
Retained message	Last retained publication stored for a topic and sent to new matching subscribers.
Persistent session	Session state that can preserve subscriptions and eligible queued messages when configured and supported.

3. Topics and subscriptions

A topic is a name used for routing messages. Mosquitto's MQTT manual states that topics do not need to be preconfigured: publishing to a topic is sufficient for the broker to route the message. A topic name is not the same thing as a REST API route.

Illustrative topic scheme

```
meters/1001/telemetry
meters/1001/status
meters/1002/telemetry
```

These are examples only; discover the actual topic names from the device, broker configuration, logs, or an authorized MQTT subscription.

Topic wildcards

Wildcard	Meaning
+	Matches one topic level. Example: meters/+/telemetry
#	Matches all remaining levels and must be the final character. Example: meters/#

Design tips

Use stable topic levels for device identity and message purpose. Avoid putting secrets or credentials in topic names. Use broker access controls to limit which clients may publish to or subscribe to each topic.

4. Publish and subscribe from the command line

The Mosquitto command-line clients are useful for testing connectivity and message flow before integrating application code.

Subscribe to one topic

```
mosquitto_sub -h BROKER_HOST -p 1883 \\
  -t "meters/1001/telemetry" -v
```

Subscribe to telemetry for multiple meters

```
mosquitto_sub -h BROKER_HOST -p 1883 \\
  -t "meters/+/telemetry" -v
```

Publish a JSON test message

```
mosquitto_pub -h BROKER_HOST -p 1883 \\
  -t "meters/1001/telemetry" \\
  -m "{\"voltage\":230.5,\"current\":5.2}"
```

Publish a retained message

```
mosquitto_pub -h BROKER_HOST -p 1883 \\
  -t "meters/1001/telemetry" -r \\
  -m "{\"voltage\":230.5,\"current\":5.2}"
```

Replace BROKER_HOST and the port with your actual MQTT listener. Add username/password or TLS options if required by your broker. Do not assume a web dashboard port is the MQTT port.

5. Quality of Service (QoS)

QoS defines the delivery assurance level for MQTT messages. Higher levels add protocol overhead. A subscriber's granted QoS can be lower than the publisher's QoS.

Level	Guarantee	Notes
0	At most once	No protocol confirmation; lowest overhead.
1	At least once	Confirmation is used; duplicates can occur.
2	Exactly once	Protocol handshake designed to deliver once; highest overhead.

For telemetry, QoS 0 may be acceptable for frequent, replaceable readings. QoS 1 is often useful when delivery matters, but application logic should tolerate duplicates. QoS does not replace database durability or application-level deduplication.

6. Retained messages, sessions, and persistence

Retained messages: When a publication is marked retained, the broker stores the latest retained message for that topic and sends it to new matching subscribers. It is a last-known-value mechanism, not a complete time-series history.

Persistent sessions: Depending on MQTT version and client/session settings, subscriptions and eligible QoS 1/2 messages can be kept for a disconnected client and delivered on reconnection.

Broker persistence: Mosquitto can save broker state to disk when configured. This is different from application-level storage of every meter reading.

> For long-term meter history, store readings in a database designed for the application's needs.

7. Broker configuration

Mosquitto is commonly configured through a mosquitto.conf file. The official configuration reference covers listener definitions, authentication, TLS, access control, persistence, bridges, and other options. The exact valid settings depend on the installed Mosquitto version.

Illustrative configuration only — do not replace an existing production configuration without reviewing it:

```
listener 1883
allow_anonymous false
password_file C:/mosquitto/config/passwords
persistence true
persistence_location C:/mosquitto/data/
```

> This example assumes Windows paths and is not a ready-to-apply configuration. Ensure the directories exist and the service account can access them. A password file does not encrypt network traffic; use TLS on untrusted networks.

Listener ports

Port 1883 is commonly used for MQTT without TLS; 8883 is commonly used for MQTT over TLS. These are conventions only. Confirm the actual listener configuration. A web dashboard may run on an entirely different port.

8. Authentication, authorization, and TLS

Authentication answers who a client is. Authorization determines which topics that client may read or write. TLS encrypts the connection and can verify the broker's identity; optional client certificates can also authenticate clients.

Security checklist

• Disable anonymous access when it is not intentionally required.

• Use separate credentials or identities for devices, backend services, and monitoring tools.

• Apply topic ACLs so clients can access only the topics they need.

• Use TLS for traffic crossing untrusted networks.

• Do not place credentials in topic names, payloads, frontend code, or source control.

• Store secrets in environment variables or a managed secret store and restrict file permissions.

• Review authentication plugin and configuration behavior for your installed Mosquitto version.

9. Backend integration pattern

A backend does not usually make an HTTP request to Mosquitto to retrieve every live message. Instead, the backend runs an MQTT client, subscribes to relevant topics, and handles messages as they arrive. The backend can cache the latest values and/or persist readings in a database. It can then expose REST endpoints for the frontend.

```
Meter/device
    -> MQTT publish
Mosquitto broker
    -> topic subscription
Backend MQTT client
    -> validate and parse payload
    -> update cache / write database
REST API
    -> frontend requests latest/history
```

Illustrative REST API design

Endpoint	Purpose
GET /api/meters	List meters known to the application.
GET /api/meters/{id}/latest	Return the latest cached or persisted reading.
GET /api/meters/{id}/history	Return readings stored in a database.
GET /api/meters/{id}/status	Return last-seen time or application-defined status.

The endpoints above are design examples, not built-in Mosquitto endpoints. MQTT topics and HTTP routes are separate interfaces.

10. Example Node.js subscriber and REST API

This minimal illustration uses the mqtt and express packages. It stores only the latest reading in process memory; data disappears when the process restarts. Use a database for persistence, add schema validation and logging, and handle reconnect/error states for production.

```
npm install mqtt express
```

```
const express = require("express");
const mqtt = require("mqtt");

const app = express();
const latest = {};
```

```
const client = mqtt.connect(process.env.MQTT_URL, {
  username: process.env.MQTT_USERNAME,
  password: process.env.MQTT_PASSWORD
});

client.on("connect", () => {
  client.subscribe("meters/+/telemetry", { qos: 1 }, (err) => {
    if (err) console.error("Subscribe failed:", err.message);
  });
});

client.on("message", (topic, payload) => {
  try {
    const parts = topic.split("/");
    const meterId = parts[1];
    const reading = JSON.parse(payload.toString());
    latest[meterId] = {
      ...reading,
      receivedAt: new Date().toISOString()
    };
  } catch (err) {
    console.error("Invalid telemetry:", err.message);
  }
});

app.get("/api/meters/:id/latest", (req, res) => {
  const reading = latest[req.params.id];
  if (!reading) {
    return res.status(404).json({ message: "No reading received yet" });
  }
  res.json(reading);
});

app.listen(3000, () => console.log("API listening on port 3000"));
```

Set MQTT_URL to the actual broker endpoint, such as mqtt://host:1883 only when that listener really uses plain MQTT. Use the correct TLS scheme and certificate configuration when applicable. Do not expose broker credentials to browsers.

11. Troubleshooting checklist

Symptom	Checks
Cannot connect	Check hostname/IP, actual listener port, firewall rules, routing, TLS mode, and broker logs.
Connection refused	The host may be reachable but no listener may be accepting connections on that port, or the service/firewall may reject it.
Not authorized	Verify credentials, ACLs, listener authentication configuration, and exact topic filter.
No messages appear	Confirm the publisher's exact topic, payload, broker address, and whether the device is publishing to this broker. Check case and slash placement.
Only see a message on connect	It may be a retained message; check whether the publisher marks messages retained.
Duplicate readings	QoS 1 can deliver duplicates; use message IDs, timestamps, or application-level deduplication if required.
Dashboard works but MQTT client fails	The dashboard's HTTP port may differ from the MQTT listener port.
Data disappears after restart	An in-memory backend cache is volatile. Configure broker persistence for broker state and a database for application history.

12. Notes for the broker shown in your screenshot

The dashboard screenshot shows the broker as online, one client online, five total subscriptions, and 56 retained/stored messages at the time shown. Those dashboard counters do not reveal the topic names, payload schema, or MQTT listener port.

> The dashboard URL uses sky:8287. Do not assume 8287 is the MQTT port; it may be the web interface port. Inspect the broker's Listeners page or the server configuration to determine the actual MQTT listener.

13. Recommended learning sequence

- Read the MQTT manual to understand topics, wildcards, QoS, retained messages, and sessions.
- Inspect the broker configuration and identify the actual listener and security settings.
- Use mosquitto_sub to verify real topic names and message payloads.
- Use mosquitto_pub to test a controlled topic with non-production data.
- Implement an MQTT subscriber in the backend and validate payloads.
- Expose REST endpoints only if an HTTP consumer such as the frontend needs them.
- Add database storage, authentication, TLS, monitoring, and operational error handling as required.

14. Official source links

Main documentation index: https://mosquitto.org/documentation/

mosquitto — broker: https://mosquitto.org/man/mosquitto-8.html

mosquitto.conf: https://mosquitto.org/man/mosquitto-conf-5.html

mosquitto_ctrl: https://mosquitto.org/man/mosquitto_ctrl-1.html

mosquitto_passwd: https://mosquitto.org/man/mosquitto_passwd-1.html

mosquitto_pub: https://mosquitto.org/man/mosquitto_pub-1.html

mosquitto_rr: https://mosquitto.org/man/mosquitto_rr-1.html

mosquitto_sub: https://mosquitto.org/man/mosquitto_sub-1.html

MQTT manual: https://mosquitto.org/man/mqtt-7.html

Authentication methods: https://mosquitto.org/documentation/authentication-methods/

Dynamic Security: https://mosquitto.org/documentation/dynamic-security/

libmosquitto API: https://mosquitto.org/api/

This guide is intended as a practical companion to the official documentation. The official website and the manual for the installed Mosquitto version remain authoritative for complete options, version-specific behavior, and updates.